

Laboratory Work 3

1. Generate and insert 200 random rows: Airline_info.

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database schema, including the 'public' schema and the 'airline' table. The main query editor contains the following SQL code:

```
268 airline_id,  
269 airline_code,  
270 airline_name,  
271 airline_country,  
272 created_at,  
273 updated_at  
274 )  
275 SELECT  
276 g,  
277 'AL' || g,  
278 'Airline_' || g,  
279 (ARRAY['Kazakhstan', 'USA', 'UK', 'Japan', 'France'])  
280 [1 + floor(random() * 5)::int],  
281 CURRENT_TIMESTAMP,  
282 CURRENT_TIMESTAMP  
283 FROM generate_series(1, 20) AS g;  
284  
285 SELECT * FROM airline_info;
```

The 'Data Output' pane shows the first 12 rows of the 'airline' table:

airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	AL1	Airline_1	UK	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
2	AL2	Airline_2	UK	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
3	AL3	Airline_3	Japan	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
4	AL4	Airline_4	France	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
5	AL5	Airline_5	USA	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
6	AL6	Airline_6	France	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
7	AL7	Airline_7	USA	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
8	AL8	Airline_8	Japan	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
9	AL9	Airline_9	Kazakhstan	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
10	AL10	Airline_10	USA	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
11	AL11	Airline_11	France	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471
12	AL12	Airline_12	UK	2026-09-29 20:19:36.119471	2026-09-29 20:19:36.119471

Query executed with server cursor. Query complete 00:00:00.148.

Airport:

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database schema, including the 'public' schema and the 'airport' table. The main query editor contains the following SQL code:

```
291 state,  
292 city,  
293 created_at,  
294 updated_at  
295 )  
296 SELECT  
297 g,  
298 'Airport_' || g,  
299 (ARRAY['Kazakhstan', 'UK', 'Japan', 'USA', 'France'])  
300 [1 + floor(random() * 5)::int],  
301 'State_' || g,  
302 (ARRAY['Astana', 'London', 'Tokyo', 'New York', 'Paris'])  
303 [1 + floor(random() * 5)::int],  
304 CURRENT_TIMESTAMP,  
305 CURRENT_TIMESTAMP  
306 FROM generate_series(1, 30) AS g;  
307  
308 SELECT * FROM airport;
```

The 'Data Output' pane shows the first 12 rows of the 'airport' table:

airport_id [PK] integer	airport_name character varying (50)	country character varying (50)	state character varying (50)	city character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	Airport_1	Kazakhstan	State_1	New York	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
2	Airport_2	France	State_2	New York	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
3	Airport_3	UK	State_3	Tokyo	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
4	Airport_4	Japan	State_4	New York	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
5	Airport_5	UK	State_5	Paris	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
6	Airport_6	UK	State_6	London	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
7	Airport_7	Japan	State_7	Paris	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
8	Airport_8	Kazakhstan	State_8	Paris	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
9	Airport_9	UK	State_9	London	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
10	Airport_10	Japan	State_10	New York	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
11	Airport_11	France	State_11	Tokyo	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294
12	Airport_12	Japan	State_12	Tokyo	2026-09-29 20:25:53.398294	2026-09-29 20:25:53.398294

Query executed with server cursor. Query complete 00:00:00.052. Successfully run. Total query runtime: 52 msec. 30 rows affected.

Passengers:

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airline', 'airport', 'baggage', 'boarding_pass', 'booking', 'booking_flight', 'flights', and 'passengers'. The 'passengers' table is selected, showing its columns: passenger_id (PK integer), first_name (character varying (50)), last_name (character varying (50)), date_of_birth (date), gender (character varying (50)), country_of_citizenship (character varying (50)), country_of_residence (character varying (50)), and passport_number (character varying (20)).

The main query editor displays the following SQL code:

```
CREATE TABLE passengers (
    passenger_id integer PRIMARY KEY,
    first_name character varying (50),
    last_name character varying (50),
    date_of_birth date,
    gender character varying (50),
    country_of_citizenship character varying (50),
    country_of_residence character varying (50),
    passport_number character varying (20),
    created_at timestamp,
    updated_at timestamp
);

INSERT INTO passengers (
    passenger_id,
    first_name,
    last_name,
    date_of_birth,
    gender,
    country_of_citizenship,
    country_of_residence,
    passport_number,
    created_at,
    updated_at
)
SELECT
    g,
    'Name_' || g,
    'Surname_' || g,
    DATE '1990-01-01' + (random() * 5000)::int,
    (ARRAY['Male', 'Female'])[1 + floor(random() * 2)::int],
    (ARRAY['Kazakhstan', 'UK', 'Japan', 'USA', 'France'])[1 + floor(random() * 5)::int],
    (ARRAY['Kazakhstan', 'UK', 'Japan', 'USA', 'France'])[1 + floor(random() * 5)::int],
    'PASS' || g,
    CURRENT_TIMESTAMP,
    CURRENT_TIMESTAMP
FROM generate_series(1, 50) AS g;
```

The 'Data Output' pane shows the first 11 rows of the 'passengers' table:

passenger_id	first_name	last_name	date_of_birth	gender	country_of_citizenship	country_of_residence	passport_number
1	Name_1	Surname_1	1997-05-01	Male	UK	Japan	PASS1
2	Name_2	Surname_2	2003-04-27	Male	Japan	USA	PASS2
3	Name_3	Surname_3	1999-06-28	Female	Kazakhstan	France	PASS3
4	Name_4	Surname_4	2001-03-29	Female	Japan	UK	PASS4
5	Name_5	Surname_5	1998-11-18	Male	UK	UK	PASS5
6	Name_6	Surname_6	2002-07-09	Female	France	USA	PASS6
7	Name_7	Surname_7	1997-02-26	Female	France	Japan	PASS7
8	Name_8	Surname_8	1995-09-02	Male	Japan	UK	PASS8
9	Name_9	Surname_9	1990-06-16	Female	Kazakhstan	Japan	PASS9
10	Name_10	Surname_10	1999-09-16	Male	Japan	UK	PASS10
11	Name_11	Surname_11	1991-04-14	Female	Kazakhstan	France	PASS11

Flights:

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays the database schema, including tables like 'airline', 'airport', 'baggage', 'boarding_pass', 'booking', 'booking_flight', 'flights', and 'passengers'. The 'flights' table is selected, showing its columns: flight_id (PK integer), sch_departure_time (timestamp without time zone), sch_arrival_time (timestamp without time zone), departing_airport_id (integer), arriving_airport_id (integer), departing_gate (text), arriving_gate (character varying (50)), and airline_id (integer).

The main query editor displays the following SQL code:

```
CREATE TABLE flights (
    flight_id integer PRIMARY KEY,
    sch_departure_time timestamp without time zone,
    sch_arrival_time timestamp without time zone,
    departing_airport_id integer,
    arriving_airport_id integer,
    departing_gate text,
    arriving_gate character varying (50),
    airline_id integer,
    created_at timestamp,
    updated_at timestamp
);

INSERT INTO flights (
    flight_id,
    sch_departure_time,
    sch_arrival_time,
    departing_airport_id,
    arriving_airport_id,
    departing_gate,
    arriving_gate,
    airline_id,
    created_at,
    updated_at
)
SELECT
    g,
    TIMESTAMP '2024-01-01 08:00:00' + g * INTERVAL '1 day',
    TIMESTAMP '2024-01-01 12:00:00' + g * INTERVAL '1 day',
    1 + floor(random() * 30)::int,
    1 + floor(random() * 30)::int,
    'Gate_' || (1 + floor(random() * 10)::int),
    'Gate_' || (1 + floor(random() * 10)::int),
    1 + floor(random() * 20)::int,
    CURRENT_TIMESTAMP,
    CURRENT_TIMESTAMP
FROM generate_series(1, 30) AS g;
```

The 'Data Output' pane shows the first 11 rows of the 'flights' table:

flight_id	sch_departure_time	sch_arrival_time	departing_airport_id	arriving_airport_id	departing_gate	arriving_gate	airline_id
1	2024-01-02 08:00:00	2024-01-02 12:00:00	23	19	Gate_3	Gate_9	17
2	2024-01-03 08:00:00	2024-01-03 12:00:00	7	1	Gate_1	Gate_8	2
3	2024-01-04 08:00:00	2024-01-04 12:00:00	11	24	Gate_4	Gate_1	9
4	2024-01-05 08:00:00	2024-01-05 12:00:00	23	2	Gate_10	Gate_4	7
5	2024-01-06 08:00:00	2024-01-06 12:00:00	15	4	Gate_7	Gate_6	6
6	2024-01-07 08:00:00	2024-01-07 12:00:00	22	13	Gate_7	Gate_5	6
7	2024-01-08 08:00:00	2024-01-08 12:00:00	26	30	Gate_3	Gate_3	10
8	2024-01-09 08:00:00	2024-01-09 12:00:00	4	15	Gate_4	Gate_8	11
9	2024-01-10 08:00:00	2024-01-10 12:00:00	23	5	Gate_8	Gate_1	2
10	2024-01-11 08:00:00	2024-01-11 12:00:00	12	7	Gate_7	Gate_1	2
11	2024-01-12 08:00:00	2024-01-12 12:00:00	3	2	Gate_8	Gate_1	12

Booking:

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the 'Object Explorer' with a tree view of the database schema. The 'booking' table is selected under the 'Tables (16)' folder. The main pane shows the 'Query' editor with a SQL query that inserts 40 rows into the 'booking' table. The query uses the 'generate_series' function to create a series of IDs and random values for other columns. The 'Data Output' pane at the bottom shows the first 12 rows of the data inserted.

```
390         updated_at,  
391         status,  
392         ticket_price  
393     )  
394     SELECT  
395         g,  
396         1 + floor(random() * 30)::int,  
397         1 + floor(random() * 50)::int,  
398         (ARRAY['Website', 'Mobile App', 'Airport Desk'])  
399             [1 + floor(random() * 3)::int],  
400         CURRENT_TIMESTAMP,  
401         CURRENT_TIMESTAMP,  
402         (ARRAY['Confirmed', 'Pending', 'Cancelled'])  
403             [1 + floor(random() * 3)::int],  
404         5000 + floor(random() * 45000)  
405     FROM generate_series(1, 40) AS g;  
406  
407     SELECT * FROM booking;
```

booking_id [PK] integer	flight_id integer	passenger_id integer	booking_platform character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	status character varying (50)	ticket_price numeric (7,2)
1	1	12	Mobile App	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Confirmed	25261.00
2	2	22	Airport Desk	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Confirmed	5899.00
3	3	15	Airport Desk	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Pending	38436.00
4	4	12	Airport Desk	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Pending	22343.00
5	5	13	Airport Desk	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Cancelled	40378.00
6	6	26	Website	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Pending	32320.00
7	7	21	Airport Desk	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Confirmed	38445.00
8	8	25	Airport Desk	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Pending	23043.00
9	9	5	Website	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Confirmed	41605.00
10	10	23	Website	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Pending	33098.00
11	11	11	Website	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Cancelled	32366.00
12	12	12	Mobile App	2026-09-29 20:40:17.388056	2026-09-29 20:40:17.388056	Pending	36657.00

Baggage_check:

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the 'Object Explorer' with a tree view of the database schema. The 'baggage_check' table is selected under the 'Tables (16)' folder. The main pane shows the 'Query' editor with a SQL query that inserts 30 rows into the 'baggage_check' table. The query uses the 'generate_series' function to create a series of IDs and random values for other columns. The 'Data Output' pane at the bottom shows the first 12 rows of the data inserted.

```
410     baggage_check_id,  
411     check_result,  
412     created_at,  
413     updated_at,  
414     booking_id,  
415     passenger_id  
416 )  
417     SELECT  
418         g,  
419         (ARRAY['Checked', 'Not checked'])  
420             [1 + floor(random() * 2)::int],  
421         TIMESTAMP '2024-03-01' + (random() * 30) * INTERVAL '1 day',  
422         CURRENT_TIMESTAMP,  
423         1 + floor(random() * 40)::int,  
424         1 + floor(random() * 50)::int  
425     FROM generate_series(1, 30) AS g;  
426  
427     SELECT * FROM baggage_check;
```

baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
1	Not checked	2024-03-29 18:03:34.356641	2026-09-29 20:42:33.670793	40	46
2	Checked	2024-03-05 09:25:55.193233	2026-09-29 20:42:33.670793	1	22
3	Checked	2024-03-20 19:04:55.751772	2026-09-29 20:42:33.670793	40	16
4	Checked	2024-03-20 10:53:48.024182	2026-09-29 20:42:33.670793	5	26
5	Checked	2024-03-16 03:22:42.296539	2026-09-29 20:42:33.670793	29	44
6	Checked	2024-03-04 19:44:36.542755	2026-09-29 20:42:33.670793	4	34
7	Checked	2024-03-09 19:19:39.436263	2026-09-29 20:42:33.670793	1	38
8	Checked	2024-03-08 23:39:10.104061	2026-09-29 20:42:33.670793	33	10
9	Checked	2024-03-27 01:12:16.201266	2026-09-29 20:42:33.670793	26	4
10	Not checked	2024-03-19 14:35:11.577884	2026-09-29 20:42:33.670793	21	48
11	Checked	2024-03-18 15:58:22.161252	2026-09-29 20:42:33.670793	2	47
12	Checked	2024-03-04 08:22:19.869451	2026-09-29 20:42:33.670793	13	29

2.

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree view of database objects, including 'airline' and 'airline_info' tables. The 'airline' table is expanded, showing its columns: 'airline_id', 'airline_code', 'airline_name', 'airline_country', 'created_at', and 'updated_at'. The 'airline_info' table is also expanded, showing its columns: 'booking_id', 'flight_id', 'passenger_id', 'booking_platform', and 'created_at'. The 'Query' pane on the right shows the following SQL code:

```
430     airline_id,  
431     airline_code,  
432     airline_name,  
433     airline_country,  
434     created_at,  
435     updated_at  
436 )  
437 VALUES (  
438     21,  
439     'KAZ',  
440     'KazAir',  
441     'Kazakhstan',  
442     CURRENT_TIMESTAMP,  
443     CURRENT_TIMESTAMP  
444 );  
445  
446 SELECT * FROM airline_info  
447 WHERE airline_name = 'KazAir';
```

The 'Data Output' pane shows the result of the query: 'INSERT 0 1'. The 'Messages' pane shows the message: 'Query returned successfully in 70 msec.'

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree view of database objects, including 'airline' and 'airline_info' tables. The 'airline' table is expanded, showing its columns: 'airline_id', 'airline_code', 'airline_name', 'airline_country', 'created_at', and 'updated_at'. The 'airline_info' table is also expanded, showing its columns: 'booking_id', 'flight_id', 'passenger_id', 'booking_platform', and 'created_at'. The 'Query' pane on the right shows the same SQL code as in the previous screenshot. The 'Data Output' pane shows the result of the query: 'INSERT 0 1'. The 'Messages' pane shows the message: 'Query returned successfully in 70 msec.'

The 'Data Output' pane also displays a table with the following data:

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
21	KAZ	KazAir	Kazakhstan	2026-09-29 20:48:43.868984	2026-09-29 20:48:43.868984

3.

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

```

437 VALUES (
438     21,
439     'KAZ',
440     'KazAir',
441     'Kazakhstan',
442     CURRENT_TIMESTAMP,
443     CURRENT_TIMESTAMP
444 );
445
446 SELECT * FROM airline_info
447 WHERE airline_name = 'KazAir';
448
449 UPDATE airline_info
450 SET airline_country = 'Turkey'
451 WHERE airline_name = 'KazAir';
452
453 SELECT * FROM airline_info
454 WHERE airline_name = 'KazAir';

```

Data Output

airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	21	KAZ	KazAir	2026-09-29 20:48:43.868984	2026-09-29 20:48:43.868984

4.

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

```

453 SELECT * FROM airline_info
454 WHERE airline_name = 'KazAir';
455
456 INSERT INTO airline_info (
457     airline_id,
458     airline_code,
459     airline_name,
460     airline_country,
461     created_at,
462     updated_at
463 )
464 VALUES
465     (22, 'AE', 'AirEasy', 'France', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
466     (23, 'FH', 'FlyHigh', 'Brazil', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
467     (24, 'FF', 'FlyFly', 'Poland', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
468
469 SELECT * FROM airline_info
470 WHERE airline_name IN ('AirEasy', 'FlyHigh', 'FlyFly');

```

Data Output

airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	22	AE	AirEasy	2026-09-29 21:02:56.669773	2026-09-29 21:02:56.669773
2	23	FH	FlyHigh	2026-09-29 21:02:56.669773	2026-09-29 21:02:56.669773
3	24	FF	FlyFly	2026-09-29 21:02:56.669773	2026-09-29 21:02:56.669773

5.

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database schema with the 'airline' table selected. The main query editor contains the following SQL code:

```

460     airline_country,
461     created_at,
462     updated_at
463 )
464 VALUES
465     (22, 'AE', 'AirEasy', 'France', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
466     (23, 'FH', 'FlyHigh', 'Brazil', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
467     (24, 'FF', 'FlyFly', 'Poland', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
468
469 SELECT * FROM airline_info
470 WHERE airline_name IN ('AirEasy', 'FlyHigh', 'FlyFly');
471
472 SELECT COUNT(*)
473 FROM flights
474 WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2024;
475
476 DELETE FROM flights
477 WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2024;

```

The 'Data Output' tab shows the result of the DELETE statement: 'DELETE 30'. A message below indicates 'Query returned successfully in 34 msec.'

6.

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database schema with the 'booking' table selected. The main query editor contains the following SQL code:

```

463 )
464 VALUES
465     (22, 'AE', 'AirEasy', 'France', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
466     (23, 'FH', 'FlyHigh', 'Brazil', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
467     (24, 'FF', 'FlyFly', 'Poland', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
468
469 SELECT * FROM airline_info
470 WHERE airline_name IN ('AirEasy', 'FlyHigh', 'FlyFly');
471
472 SELECT COUNT(*)
473 FROM flights
474 WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2024;
475
476 DELETE FROM flights
477 WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2024;
478
479 UPDATE booking
480 SET ticket_price = ticket_price * 1.15;

```

The 'Data Output' tab shows the result of the UPDATE statement: 'UPDATE 40'. A message below indicates 'Query returned successfully in 56 msec.'

7.

The screenshot shows the PostgreSQL Object Explorer on the left, with the 'booking' table selected under the 'airline' schema. The 'Columns (8)' list is expanded, showing: booking_id integer, flight_id integer, passenger_id integer, booking_platform, created_at timestamp, and updated_at timestamp. The main window displays the Query Editor with the following SQL code:

```

483 FROM booking;
484
485 SELECT COUNT(*)
486 FROM booking
487 WHERE ticket_price < 10000;
488
489 DELETE FROM booking
490 WHERE ticket_price < 10000;
491
492 DELETE FROM baggage_check
493 WHERE booking_id IN (
494     SELECT booking_id
495     FROM booking
496     WHERE ticket_price < 10000
497 );
498
499 DELETE FROM booking
500 WHERE ticket_price < 10000;

```

The 'Data Output' tab shows the result of the DELETE query: 'DELETE 3'. The 'Messages' tab shows: 'Query returned successfully in 41 msec.'

8.

The screenshot shows the PostgreSQL Object Explorer on the left, with the 'airline_info' table selected under the 'airline' schema. The 'Columns (8)' list is expanded, showing: booking_id integer, flight_id integer, passenger_id integer, booking_platform, created_at timestamp, and updated_at timestamp. The main window displays the Query Editor with the following SQL code:

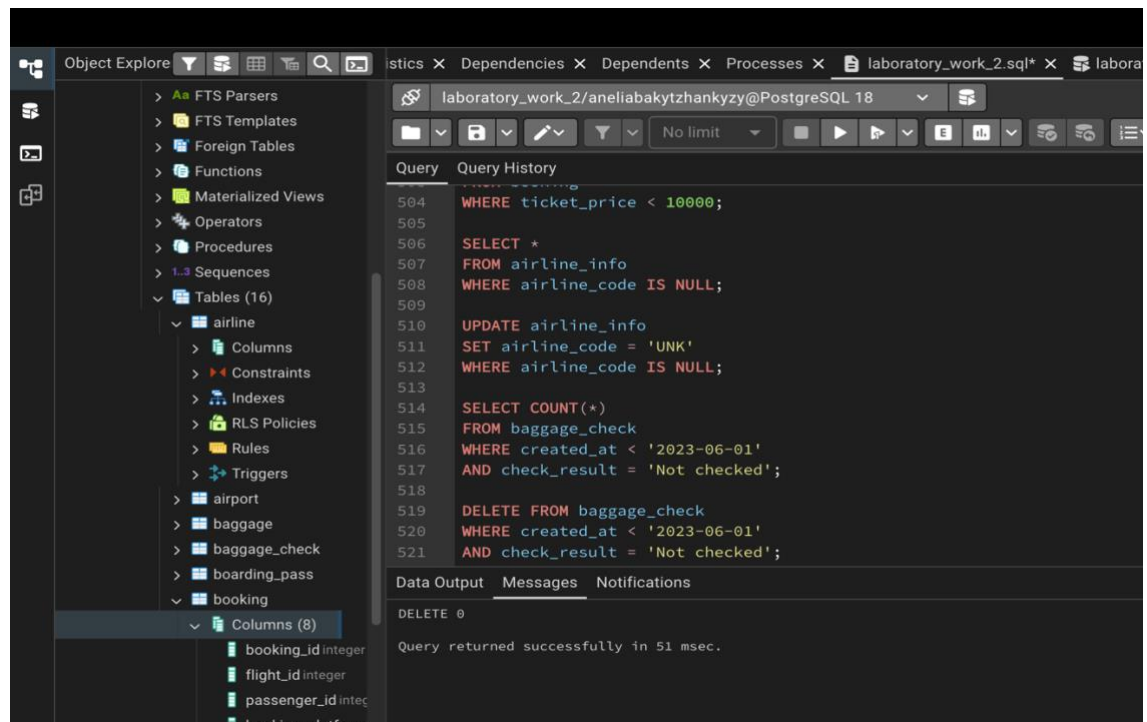
```

495 FROM booking
496 WHERE ticket_price < 10000
497 );
498
499 DELETE FROM booking
500 WHERE ticket_price < 10000;
501
502 SELECT COUNT(*)
503 FROM booking
504 WHERE ticket_price < 10000;
505
506 SELECT *
507 FROM airline_info
508 WHERE airline_code IS NULL;
509
510 UPDATE airline_info
511 SET airline_code = 'UNK'
512 WHERE airline_code IS NULL;

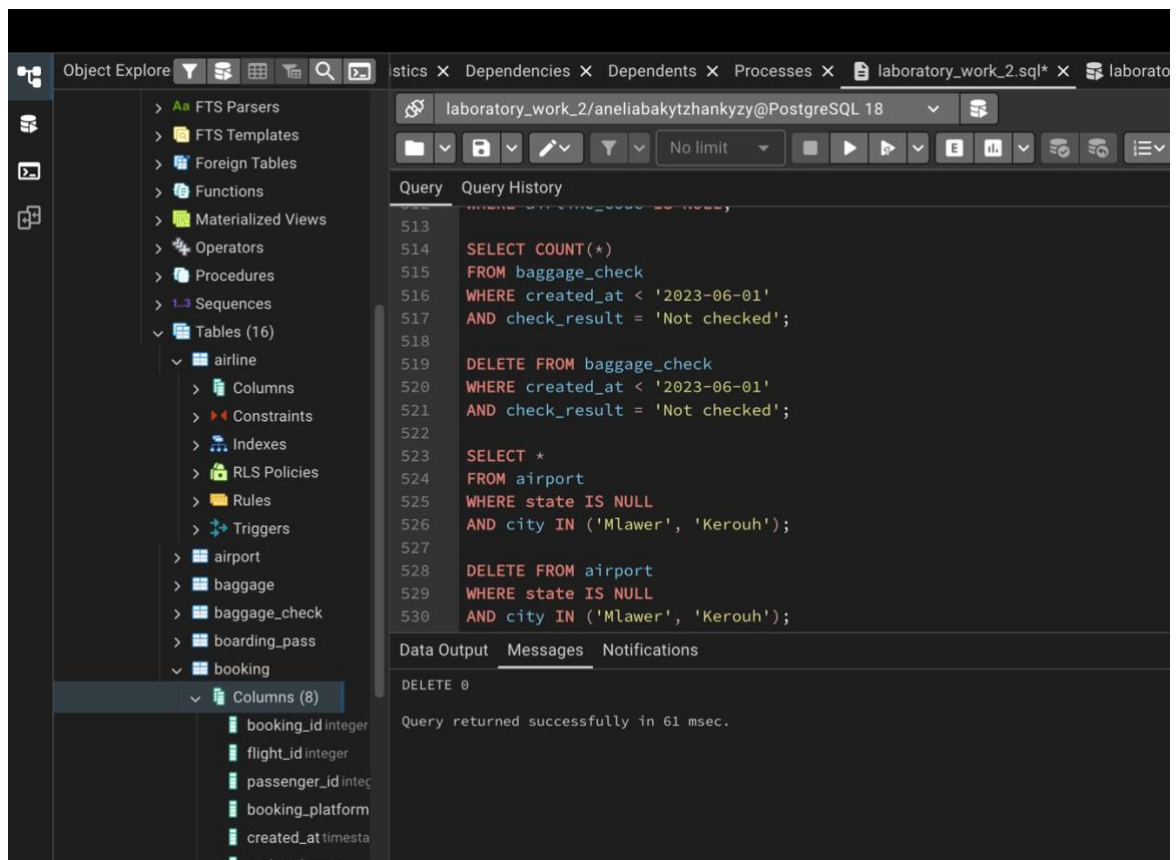
```

The 'Data Output' tab shows the result of the UPDATE query: 'UPDATE 0'. The 'Messages' tab shows: 'Query returned successfully in 46 msec.'

9.



10.



11.

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database schema, including tables like `airline`, `airport`, `baggage`, `baggage_check`, `boarding_pass`, and `booking`. The main editor displays an SQL query:

```

547
548 INSERT INTO baggage_check (
549     baggage_check_id,
550     check_result,
551     created_at,
552     updated_at,
553     booking_id,
554     passenger_id
555 )
556 VALUES (
557     31,
558     'Not checked',
559     CURRENT_TIMESTAMP,
560     CURRENT_TIMESTAMP,
561     1,
562     22
563 )
564 RETURNING baggage_check_id, created_at;

```

The bottom panel shows the 'Data Output' tab with a single row of data:

baggage_check_id [PK] integer	created_at timestamp without time zone
1	2026-09-29 22:34:23.588656

12.

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database schema, including tables like `airline`, `airport`, `baggage`, `baggage_check`, `boarding_pass`, and `booking`. The main editor displays an SQL query:

```

552     updated_at,
553     booking_id,
554     passenger_id
555 )
556 VALUES (
557     31,
558     'Not checked',
559     CURRENT_TIMESTAMP,
560     CURRENT_TIMESTAMP,
561     1,
562     22
563 )
564 RETURNING baggage_check_id, created_at;
565
566 UPDATE airline_info
567 SET airline_country = UPPER(airline_country);
568 SELECT airline_id, airline_name, airline_country
569 FROM airline_info;

```

The bottom panel shows the 'Data Output' tab with a table of 12 rows:

airline_id [PK] integer	airline_name character varying (50)	airline_country character varying (50)
1	Airline_1	UK
2	Airline_2	UK
3	Airline_3	JAPAN
4	Airline_4	FRANCE
5	Airline_5	USA
6	Airline_6	FRANCE
7	Airline_7	USA
8	Airline_8	JAPAN
9	Airline_9	KAZAKHSTAN
10	Airline_10	USA
11	Airline_11	FRANCE
12	Airline_12	UK

At the bottom, a status bar indicates: "Query executed with server cursor" and "Query complete 00:00:00.058".

13.

Object Explorer

- FTS Parsers
- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (16)
 - airline
 - Columns
 - Constraints
 - Indexes
 - RLS Policies
 - Rules
 - Triggers
 - airport
 - baggage
 - baggage_check
 - boarding_pass
 - booking
 - Columns (8)
 - booking_id integer
 - flight_id integer
 - passenger_id integer
 - booking_platform
 - created_at timestamp
 - updated_at timestamp
 - status character varying (50)
 - ticket_price numeric

Query

```

561     1,
562     22
563 )
564 RETURNING baggage_check_id, created_at;
565
566 UPDATE airline_info
567 SET airline_country = UPPER(airline_country);
568 SELECT airline_id, airline_name, airline_country
569 FROM airline_info;
570
571 UPDATE airline_info
572 SET airline_name = 'Global Airways',
573     airline_country = 'United Kingdom',
574     updated_at = CURRENT_TIMESTAMP
575 WHERE airline_id = 5;
576 SELECT airline_id, airline_name, airline_country, updated_at
577 FROM airline_info
578 WHERE airline_id = 5;
  
```

Data Output

airline_id [PK] integer	airline_name character varying (50)	airline_country character varying (50)	updated_at timestamp without time zone	
1	5	Global Airways	United Kingdom	2026-09-29 22:37:44.386581

Showing rows: 1 to 1

14.

Object Explorer

- FTS Parsers
- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (16)
 - airline
 - Columns
 - Constraints
 - Indexes
 - RLS Policies
 - Rules
 - Triggers
 - airport
 - Columns (8)
 - airport_id integer
 - airport_name character varying (50)
 - city character varying (50)
 - state character varying (50)
 - baggage
 - baggage_check
 - boarding_pass
 - booking
 - Columns (8)
 - booking_id integer
 - flight_id integer
 - passenger_id integer
 - booking_platform
 - created_at timestamp
 - updated_at timestamp
 - status character varying (50)
 - ticket_price numeric
 - Constraints
 - Indexes
 - RLS Policies
 - Rules
 - Triggers
 - booking_flight

Query

```

568 SELECT airline_id, airline_name, airline_country
569 FROM airline_info;
570
571 UPDATE airline_info
572 SET airline_name = 'Global Airways',
573     airline_country = 'United Kingdom',
574     updated_at = CURRENT_TIMESTAMP
575 WHERE airline_id = 5;
576 SELECT airline_id, airline_name, airline_country, updated_at
577 FROM airline_info
578 WHERE airline_id = 5;
579
580 UPDATE airport
581 SET state = 'Capital District'
582 WHERE city IN ('Astana', 'London', 'Tokyo');
583 SELECT airport_id, airport_name, city, state
584 FROM airport
585 WHERE city IN ('Astana', 'London', 'Tokyo');
  
```

Data Output

airport_id [PK] integer	airport_name character varying (50)	city character varying (50)	state character varying (50)
1	3	Airport_3	Tokyo
2	6	Airport_6	London
3	9	Airport_9	London
4	11	Airport_11	Tokyo
5	12	Airport_12	Tokyo
6	13	Airport_13	London
7	15	Airport_15	Tokyo
8	16	Airport_16	Astana
9	17	Airport_17	Tokyo
10	18	Airport_18	London
11	19	Airport_19	Astana
12	20	Airport_20	Astana

Showing rows: 1 to 18

Query executed with server cursor | Query complete 00:00:00.054

15.

The screenshot displays a PostgreSQL database management tool interface. On the left, the 'Object Explorer' pane shows a hierarchical view of the database schema. The 'airline' table is selected, and its columns are listed: `booking_id` (integer), `flight_id` (integer), `passenger_id` (integer), `booking_platform`, and `created_at` (timestamp). The main query editor on the right shows a SQL query with line numbers 580 through 597. The query consists of two parts: an `UPDATE` statement for the `airport` table and a `SELECT` statement for the `baggage_check` table. The `UPDATE` statement sets the `state` to 'Capital District' for airports in 'Astana', 'London', or 'Tokyo'. The `SELECT` statement counts the number of baggage checks created between '2024-03-01' and '2024-04-01' where the `check_result` is 'Not checked'. Below the query editor, the 'Data Output' pane shows the result of the `UPDATE` statement: 'UPDATE 11' and a message 'Query returned successfully in 44 msec.'

```
580 UPDATE airport
581 SET state = 'Capital District'
582 WHERE city IN ('Astana', 'London', 'Tokyo');
583 SELECT airport_id, airport_name, city, state
584 FROM airport
585 WHERE city IN ('Astana', 'London', 'Tokyo');
586
587 SELECT COUNT(*)
588 FROM baggage_check
589 WHERE created_at >= '2024-03-01'
590 AND created_at < '2024-04-01'
591 AND check_result = 'Not checked';
592
593 UPDATE baggage_check
594 SET check_result = 'Checked'
595 WHERE created_at >= '2024-03-01'
596 AND created_at < '2024-04-01'
597 AND check_result = 'Not checked';
```

UPDATE 11
Query returned successfully in 44 msec.